

Санкт-Петербургский государственный университет

Программная инженерия

Группа 24.М71-мм

Разработка и оптимизация системы голосового управления для автоматизированного заполнения медицинской документации

Альшаеб Басель

Отчёт по производственной практике
в форме «Решение»

Научный руководитель:
ст. преподаватель кафедры системного программирования, к.ф.-м.н. С. С. Сысоев.

Санкт-Петербург
2026

Оглавление

Список использованных сокращений	4
Введение	6
1. Постановка задачи	9
2. Обзор существующих решений	11
2.1. Выделение участков речи: Voice Activity Detection	11
2.2. Активация голосового режима: wake word / keyword spotting	12
2.3. Распознавание речи (ASR) и выбор сервиса	14
2.4. Идентификация команд в распознанном тексте: бинарная классификация	14
2.5. Интерпретация команд и ограничение допустимых действий	15
2.6. Обзор используемых технологий	16
2.7. Выводы	16
3. Архитектура подсистемы голосового управления	19
3.1. Компонентная модель	19
3.2. Сценарий взаимодействия	20
3.3. Модуль интеллектуальной обработки команд	21
4. Особенности реализации	23
4.1. Клиентский модуль: захват аудио и VAD	23
4.2. Клиентский модуль: выделение сегментов речи	24
4.3. Wake word: варианты реализации и выбранный подход .	24
4.4. Серверный модуль: приём аудио, ASR и интерпретация .	25
4.5. Серверная идентификация команд по транскрипции . . .	26
4.6. Ограничение вывода языковой модели и валидация результата	27
4.7. Интеграция с МИС «СТОММИС» и особенности предметной области	31

4.8. Нефункциональные требования: ресурсы	31
5. Интерпретация команд и заполнение медицинской карты	32
5.1. Разделение речевых фрагментов и управляющих команд	33
5.2. Формирование структурного представления команды . .	33
5.3. Использование языковой модели для интерпретации команд	34
5.4. Ограничение пространства решений и фильтрация команд	34
5.5. Особенности заполнения стоматологической карты . . .	35
5.6. Интеграция с медицинской информационной системой .	35
5.7. Сравнение с традиционным подходом	36
5.8. Выводы по главе	36
6. Эксперимент	37
6.1. Исследовательские вопросы	37
6.2. Метрики	37
6.3. Сценарии и базовая линия сравнения	38
6.4. Результаты и обсуждение	38
6.5. Обсуждение результатов	40
6.6. Угрозы нарушения корректности	41
7. Заключение	44
Список литературы	46

Список использованных сокращений

API *Application Programming Interface* — программный интерфейс взаимодействия компонентов.

ASR *Automatic Speech Recognition* — автоматическое распознавание речи (преобразование аудио в текст).

CPU *Central Processing Unit* — центральный процессор (в работе подчёркивается возможность выполнения моделей на CPU без GPU).

DeepSeek семейство больших языковых моделей (LLM), рассматриваемое как перспектива для экспериментов в задаче интерпретации команд.

DTW *Dynamic Time Warping* — метод сравнения последовательностей (используется в некоторых подходах keyword spotting).

Ffmpeg набор утилит для обработки мультимедиа (в работе используется для транскодирования аудио на сервере).

FP/FN *False Positive / False Negative* — ложное срабатывание / пропуск (например, при детекции wake word).

Intent *intent* — **намерение/тип команды** в задачах обработки естественного языка (класс действия, которое должна выполнить система).

JSON *JavaScript Object Notation* — формат обмена структурированными данными.

KWS *Keyword Spotting* — обнаружение ключевого слова (wake word).

LLM *Large Language Model* — большая языковая модель, используемая для семантической интерпретации команд.

MFCC *Mel-Frequency Cepstral Coefficients* — кепстральные коэффициенты, часто используемые как признаки в задачах аудио.

NLP *Natural Language Processing* — обработка естественного языка.

МИС медицинская информационная система.

МИС «СТОММИС» медицинская информационная система «СТОММИС» (объект внедрения в работе).

OGG контейнерный формат Ogg (в работе используется Ogg Opus для унификации входа ASR).

Opus аудиокодек Opus (в работе используется при транскодировании в Ogg Opus).

ONNX *Open Neural Network Exchange* — формат представления нейросетевых моделей.

ONNX Runtime библиотека выполнения моделей в формате ONNX (используется в клиентском VAD).

RMS *Root Mean Square* — среднеквадратичная амплитуда сигнала (простой признак для пороговой детекции речи).

Slot *slot* — **параметр команды** (атрибут/значение, извлекаемое из текста команды; например номер зуба, диагноз, процедура).

VAD *Voice Activity Detection* — детекция речевой активности (определение участков речи и «тишины»).

VoIP Voice over IP — передача голоса по IP-сетям (телефония через Интернет).

WASM *WebAssembly* — формат и среда выполнения кода в браузере (используется для ускорения вычислений на клиенте).

WebRTC набор технологий реального времени в вебе; включает классические алгоритмы VAD и подавления эха.

wake word / trigger word ключевое слово/фраза активации голосового режима.

Введение

Цифровизация медицинских учреждений приводит к тому, что часть клинической работы переносится в информационные системы: врач фиксирует жалобы, анамнез, результаты осмотра и назначений в электронных формах. В стоматологии это особенно заметно: итогом приёма является структурированная «карта пациента» (диагнозы, манипуляции, зубная формула, рекомендации), которая должна быть заполнена быстро и без потери качества. В рамках эксплуатации медицинской информационной системы (МИС) «СТОММИС» было выявлено, что взаимодействие врача с компьютером непосредственно *в процессе лечения* крайне неудобно: руки заняты инструментами, врач вынужден постоянно переключать внимание между пациентом и интерфейсом. На практике это приводит к тому, что значимая часть заполнения карты откладывается на конец приёма, что увеличивает суммарное время приёма и повышает риск потери деталей из-за человеческого фактора.

Существующие решения на рынке нередко сводятся к диктовке «сплошного текста» в аудио/текстовый протокол или к полной записи консультации с последующей расшифровкой. Подходы такого типа уменьшают объём ручного ввода, но не решают задачу *структурирования* данных (заполнение конкретных полей карты) и создают дополнительную нагрузку: постоянное распознавание речи требует вычислительных ресурсов и, при использовании облачных сервисов, приводит к росту стоимости обработки и сетевого трафика. Кроме того, при непрерывной обработке возрастают риски ложных срабатываний на разговоры пациента и фоновые звуки.

Целью данной работы является разработка и экспериментальная оценка прототипа подсистемы голосового управления для МИС «СТОММИС», обеспечивающей выделение участков речи и «тишины» в аудиопотоке, захватываемом в браузере, и последующую обработку голосовых команд, соответствующих ограниченному набору допустимых действий в МИС «СТОММИС». При этом система должна быть ресурсоэффективной и не выполнять постоянное распознавание всего потока.

Ключевые подзадачи, сформулированные руководителем, включают:

1. Определение участков с речью и без речи (условной «тишиной»)
2. Выделение в речи команд из короткого списка (при наличии технической возможности и приемлемого качества)
3. Внесение изменений в карту пациента в формате МИС «СТОМ-МИС»

В ходе исследования были последовательно проверены три способа запуска командного режима. Первоначально в прототипе использовалась ручная активация (кнопка/горячая клавиша *push-to-talk*), однако такой сценарий плохо применим во время лечения. Далее был рассмотрен вариант голосовой активации по ключевому слову (*wake word*), но практические испытания показали, что необходимость регулярно произносить слово-триггер раздражает врача и снижает применимость решения. В итоговом варианте прототипа запуск дорогостоящей обработки выполняется автоматически: после распознавания речи (ASR) на сервере применяется бинарный классификатор, определяющий, является ли транскрипция командой для МИС «СТОММИС» или речевым мусором. Только для фраз, классифицированных как команды, выполняются семантическая интерпретация (LLM) и внесение изменений в карту пациента. Выбор бинарной постановки обусловлен тем, что задача фильтрации «команда/мусор» существенно проще полной интерпретации, а её можно решать быстро и устойчиво, уменьшая число обращений к дорогостоящей LLM-обработке.

Предлагаемый подход основан на многоэтапной обработке. На стороне клиента в браузере непрерывно работает лёгкий модуль VAD, который определяет начало и конец речевого сегмента и завершает его по правилу «конец высказывания — 2 секунды тишины». На сервер передаются только выделенные VAD сегменты речи, что уменьшает сетевую нагрузку и потенциальную стоимость распознавания. На сервере аудиофрагмент при необходимости транскодируется, затем

распознаётся с использованием сервиса Yandex SpeechKit (ASR). После получения транскрипции применяется бинарный классификатор *command/not-command*, который решает, требуется ли дальнейшая обработка. Только для фраз, классифицированных как команды, выполняются семантическая интерпретация (LLM) при строгом ограничении допустимых действий и внесение изменений в карту пациента в формате МИС «СТОММИС» посредством REST API. Прототип апробировался в том числе в условиях реальной стоматологической поликлиники СПб ГБУЗ «СП №8».

1. Постановка задачи

Целью работы является разработка прототипа подсистемы голосового управления для МИС «СТОММИС», позволяющей врачу работать с медицинской картой *в процессе лечения* с помощью голоса, без необходимости использовать клавиатуру/мышь и без сценария обязательного нажатия кнопки *push-to-talk*. При этом решение должно быть ресурсоэффективным: не выполнять постоянное распознавание всего аудиопотока и минимизировать сетевой трафик.

Для достижения цели были поставлены следующие задачи.

1. Провести обзор существующих решений в области голосового управления и выделения команд в медицинских информационных системах.
2. Спроектировать архитектуру подсистемы голосового управления для МИС «СТОММИС», определить компоненты: браузерный клиент, сервер обработки, сервис распознавания речи, модуль интерпретации, интерфейс интеграции с МИС, границы ответственности и основной сценарий взаимодействия.
3. Разработать клиентский модуль захвата аудиопотока в браузере с выделением участков с речью/без речи (VAD) в реальном времени (раздел 4.1).
4. Интегрировать обработку аудиосегментов с серверным контуром: приём аудиофайла, транскодирование при необходимости, распознавание речи (ASR) с использованием сервиса распознавания речи Yandex SpeechKit и интерпретация распознанного текста в структурированное действие для заполнения полей карты пациента в МИС «СТОММИС» (раздел 4.4).
5. Реализовать механизм выделения команд без необходимости постоянного произнесения слова активации: после распознавания речи (ASR) применять на сервере бинарный классификатор

command/not-command, определяющий, требуется ли запуск дальнейшей обработки (LLM и выполнение действия в МИС «СТОМ-МИС») (раздел 4.5).

6. Провести экспериментальную валидацию прототипа, включая апробацию в реальных условиях (СПб ГБУЗ «СП №8»): оценить задержку, потребление ресурсов, частоту ложных срабатываний/пропусков и потенциальное снижение затрат по сравнению с вариантом постоянного распознавания аудиопотока (раздел 6).

2. Обзор существующих решений

В данном разделе приведён обзор подходов и инструментов, применимых для построения голосового управления в медицинских информационных системах. Основной акцент сделан на практических аспектах: как выделять речевые сегменты в аудиопотоке (VAD), как отбирать из них только команды и запускать дорогостоящую обработку (ASR/LLM) только при необходимости, а также как преобразовывать распознанный текст в контролируемые действия внутри МИС.

2.1. Выделение участков речи: Voice Activity Detection

Задача детекции речевой активности (VAD) заключается в определении моментов начала и окончания речи в непрерывном аудиопотоке. Простейший подход основан на пороговой оценке энергии сигнала (например, RMS) и может быть реализован непосредственно в браузере. Преимущество такого решения — минимальная вычислительная сложность. Недостатки проявляются в реальных условиях кабинета: фоновые шумы, переменная громкость голоса и короткие паузы приводят к нестабильным границам сегментов и к росту числа ложных срабатываний.

Более устойчивый класс решений — классический VAD, применяемый в системах связи (например, VAD из стека WebRTC) [11]. Он широко применяется в VoIP и видеоконференциях и рассчитан на работу в реальном времени.

В последние годы получили распространение нейросетевые VAD-модели, обученные на больших корпусах аудио. К популярным открытым решениям относится *Silero VAD* [9], обеспечивающий высокую точность и работающий на CPU. Недостатком является то, что типовой сценарий использования *Silero VAD* предполагает серверное выполнение в Python. Запуск на стороне клиента возможен, но в этом случае требуется самостоятельно собрать и поддерживать

цепочку: экспорт модели в ONNX, подключение ONNX Runtime Web и WebAssembly-ассетов, приведение частоты дискретизации, а также контроль производительности в браузере. Для дипломного прототипа это избыточно по трудозатратам по сравнению с готовым браузерным решением.

В рамках дипломной работы для браузера был выбран open-source инструмент @ricky0123/vad-web [8], который применяет нейросетевую модель (в формате ONNX) и исполняет её через ONNX Runtime Web [7]. Данный вариант обеспечивает более устойчивое выделение речи по сравнению с пороговой энергией и допускает работу «на месте» (в браузере) без постоянных серверных вычислений.

2.2. Активация голосового режима: wake word / keyword spotting

Чтобы врач мог использовать голосовые команды *в процессе лечения* без нажатия кнопок, требуется механизм активации голосового режима по ключевому слову (wake word). В литературе и практике встречаются два основных класса решений:

- **Локальное keyword spotting (KWS):** ключевое слово детектируется на клиенте по аудиопризнакам (например, MFCC) с использованием DTW-сопоставления «динамическое выравнивание по времени» (DTW) или небольшой нейросети.
- **Серверная проверка ключевого слова через ASR:** на клиенте выделяются короткие сегменты речи (VAD), после чего на сервер отправляется короткий фрагмент для распознавания, и активация выполняется по текстовой транскрипции.

Плюсом локального KWS является минимальная задержка и отсутствие сетевых обращений в режиме ожидания. Минусы проявляются в прикладном сценарии стоматологии: существенно меняются микрофоны/акустика, вариативны голоса, а обучение на каждого врача (персонализация) ухудшает удобство внедрения. Среди open-source реше-

ний KWS можно отметить проект *open Wake Word* [1], ориентированный на CPU-исполнение. Однако перенос KWS непосредственно в браузер требует воспроизводимой цепочки обработки аудио: ресэмплирование до фиксированной частоты (например, 16 кГц), разбиение на фреймы фиксированного размера, применение окна, вычисление спектра (FFT) и извлечение признаков (например, MFCC) с одинаковой нормализацией. На практике даже небольшие расхождения в размерах буфера/FFT или в нормализации приводят к росту доли пропусков (ложных отрицаний, false negatives) и требуют отдельной калибровки под конкретный микрофон и голос.

Серверная проверка ключевого слова через ASR проще интегрируется в существующий серверный контур распознавания речи. Кроме того, этот подход позволяет обойти необходимость обучения на каждого врача, однако он увеличивает число обращений к ASR по сравнению с чисто локальным KWS.

В дипломной работе исследовались оба варианта (см. раздел 4.3): клиентский вариант был реализован как прототип, но не прошёл испытания из-за высокой доли пропусков (false negatives) и из-за необходимости обучения под конкретного врача; в итоговой версии используется серверная проверка ключевого слова на основе ASR.

Практика показала, что регулярное использование wake word в длительном приёме неудобно для врача: необходимость повторять слово-триггер снижает естественность работы и воспринимается как дополнительная нагрузка. Поэтому в итоговом варианте прототипа wake word рассматривается как контрольный режим и резервный сценарий, а основной рабочий подход использует автоматическую идентификацию команд по транскрипции после ASR с помощью бинарного классификатора *command/not-command*.

В ходе работы последовательно исследовались несколько способов запуска обработки команд: ручная активация, активация по ключевому слову и автоматическая фильтрация команд по транскрипции после ASR. Их сравнительные характеристики приведены в табл. 1.

Как видно из табл. 1, бинарная классификация транскрипции по-

сле ASR обеспечивает более естественный сценарий использования по сравнению с wake word и при этом позволяет ограничить число дорогостоящих вызовов LLM. По этой причине именно данный подход выбран в качестве основного в итоговой версии прототипа.

2.3. Распознавание речи (ASR) и выбор сервиса

После выделения аудиосегмента система должна получить текстовую транскрипцию команды. В практических системах используются как локальные движки распознавания (например, Vosk), так и облачные сервисы. В данной работе выбран сервис Yandex SpeechKit [12], поскольку он предоставляет готовый API для русского языка.

2.4. Идентификация команд в распознанном тексте: бинарная классификация

Использование только VAD недостаточно: в выделенных сегментах речи встречаются реплики, не предназначенные для обработки системой (общение с пациентом, комментарии, уточнения, фоновые фразы). Чтобы запуск дорогостоящих этапов (LLM и внесение данных в МИС) происходил только для команд, применяется дополнительный фильтр после ASR.

В работе используется бинарный классификатор *command/not-command*, работающий по текстовой транскрипции. Такой подход имеет практические преимущества.

- врачу не требуется регулярно произносить слово активации, что повышает удобство использования;
- фильтрация после ASR проще и устойчивее локального keyword spotting в браузере (нет зависимости от микрофона/FFT/MFCC цепочки);
- LLM вызывается только для фраз, классифицированных как команды, что снижает вычислительные затраты и стоимость обра-

ботки.

Для выбора порога решения приоритетом является высокая полнота (recall) класса *command*, чтобы минимизировать пропуски команд врача; допускается некоторое увеличение числа ложных срабатываний, поскольку ошибки далее отсекаются валидатором и логикой применения в МИС.

2.5. Интерпретация команд и ограничение допустимых действий

В рамках данной работы для интерпретации команд рассматриваются облачные LLM-модели; в прототипе используется `qwen3-235b-a22b-fp8`, также исследуются альтернативы `gpt-oss-20b`. На этапе предварительного анализа также рассматривались YandexGPT и (в перспективе) DeepSeek; однако в основной конфигурации прототипа зафиксирована модель `qwen3-235b-a22b-fp8`.

Полученный текст команды необходимо преобразовать в действие внутри МИС. Классические подходы включают грамматические правила и классификацию интенгов (англ. *intent*) и параметров-слотов (англ. *slots*) для фиксированного набора команд. Однако естественная речь врача вариативна, поэтому жёсткие шаблоны быстро усложняются и требуют постоянной поддержки.

Перспективный подход — использование языковой модели (LLM) для семантической интерпретации команды. При этом критически важно ограничить допустимый вывод модели, чтобы исключить неконтролируемые действия. В прикладных системах это достигается комбинацией:

- строгого формата ответа (например, JSON) и серверной валидации;
- ограниченного справочника действий (список допустимых интенгов) и параметров (слоты, значения из справочников);

- повторного запроса к модели при нарушении формата или при низкой уверенности.

2.6. Обзор используемых технологий

Захват аудио в браузере. Для доступа к микрофону применяется API `MediaDevices.getUserMedia` [4], а для потоковой обработки и/или записи — Web Audio API [6] и MediaRecorder API [5]. Указанные технологии доступны во всех современных браузерах и не требуют установки дополнительного ПО на рабочее место врача.

Браузерный VAD. В качестве VAD-ядра выбран `@ricky0123/vad-web` [3]. Данный вариант обеспечивает баланс между простотой интеграции (JS, события `onSpeechStart/onSpeechEnd`) и ресурсной эффективностью (WebAssembly + оптимизированные вычисления ONNX Runtime Web [7]). В работе используется завершение сегмента после паузы фиксированной длительности (например, 2 секунды), что соответствует требованиям к выделению команд.

Серверная обработка и интеграция. Серверный контур реализован на Python (Flask) и принимает аудио сегменты как файл `multipart/form-data`. При необходимости входной формат приводится к Ogg Opus с помощью `ffmpeg` [2], после чего аудио подаётся в модуль распознавания речи (Yandex SpeechKit [12]).

Затем на серверной стороне после получения транскрипции применяется фильтрация команд: бинарный классификатор *command/not-command* определяет, требуется ли дальнейшая обработка. Только после решения *command* текст передаётся в модуль интерпретации (правила/LLM), который возвращает структурированное действие для заполнения полей карты пациента.

2.7. Выводы

По результатам обзора для прототипа выбрана архитектура *VAD в браузере + ASR на сервере + фильтрация команд по транскрипции + интерпретация и внесение данных*. Она позволяет:

- снизить стоимость и сетевую нагрузку за счёт отправки на сервер только речевых сегментов;
- сохранить качество распознавания русской речи благодаря использованию готового ASR-сервиса;
- минимизировать требования к рабочему месту врача, ограничившись браузером и микрофоном;
- уменьшить число обращений к LLM за счёт серверной фильтрации *command/not-command* по транскрипции ASR.

Таблица 1: Сравнение исследованных способов запуска обработки команд

Подход	Преимущества	Недостатки	Статус в работе
Ручная активация (<i>push-to-talk</i>)	Простота реализации; предсказуемый момент начала команды; удобен как контрольный сценарий	Требует использования клавиатуры/мыши или горячей клавиши; плохо применим во время лечения, когда руки заняты	Использовался как базовый и контрольный сценарий
Активация по ключевому слову (<i>wake word</i>)	Позволяет работать без нажатия кнопок; естественнее, чем ручной запуск	Требует регулярного повторения слова активации; клиентский вариант даёт высокий процент пропусков; локальный KWS требует калибровки под врача/микрофон	Исследован, но не выбран как основной режим
Бинарная классификация транскрипции после ASR (<i>command/not-command</i>)	Не требует постоянного произнесения слова активации; уменьшает число вызовов LLM; позволяет отбрасывать речевой мусор перед дорогой обработкой	Требует размеченного набора данных и обучения модели; качество зависит от ошибок ASR	Выбранный основной подход

Клиент (браузер): Web Audio API → VAD → сегмент речи → HTTP multipart/form-data (file) Сервер (Python): приём file → (FFmpeg) → ASR (Yandex SpeechKit) → классификация транскрипции <i>command/not-command</i> → (если <i>command</i>) интерпретация (правила/LLM) → валидация → действие в МИС (REST API)

Рис. 1: Конвейер обработки голосовой команды

3. Архитектура подсистемы голосового управления

В данном разделе описывается архитектура подсистемы голосового управления для МИС «СТОММИС». Архитектура фиксирует состав компонентов, распределение ответственности и основной сценарий обработки голосовой команды. На уровне проектирования система разделена на клиентскую часть (браузер) и серверный контур обработки, а также на внешние сервисы распознавания речи и модуль интеллектуальной обработки команд.

3.1. Компонентная модель

На рис. 2 приведена компонентная диаграмма. Браузерный клиент включает модули захвата аудиопотока (Web Audio API) и выделения речевых сегментов (VAD), а также модуль передачи сегментов на сервер. Серверный контур реализован в виде веб-сервиса (Flask) и включает приём аудиофайла, транскодирование (при необходимости), распознавание речи (ASR), бинарную классификацию транскрипции *command/not-command* и интерпретацию команды. Классификатор используется как «дешёвый фильтр», который предотвращает запуск LLM для нерелевантных фраз и тем самым снижает нагрузку и стоимость обработки. Внесение изменений в карту пациента выполняется посредством REST API МИС «СТОММИС».

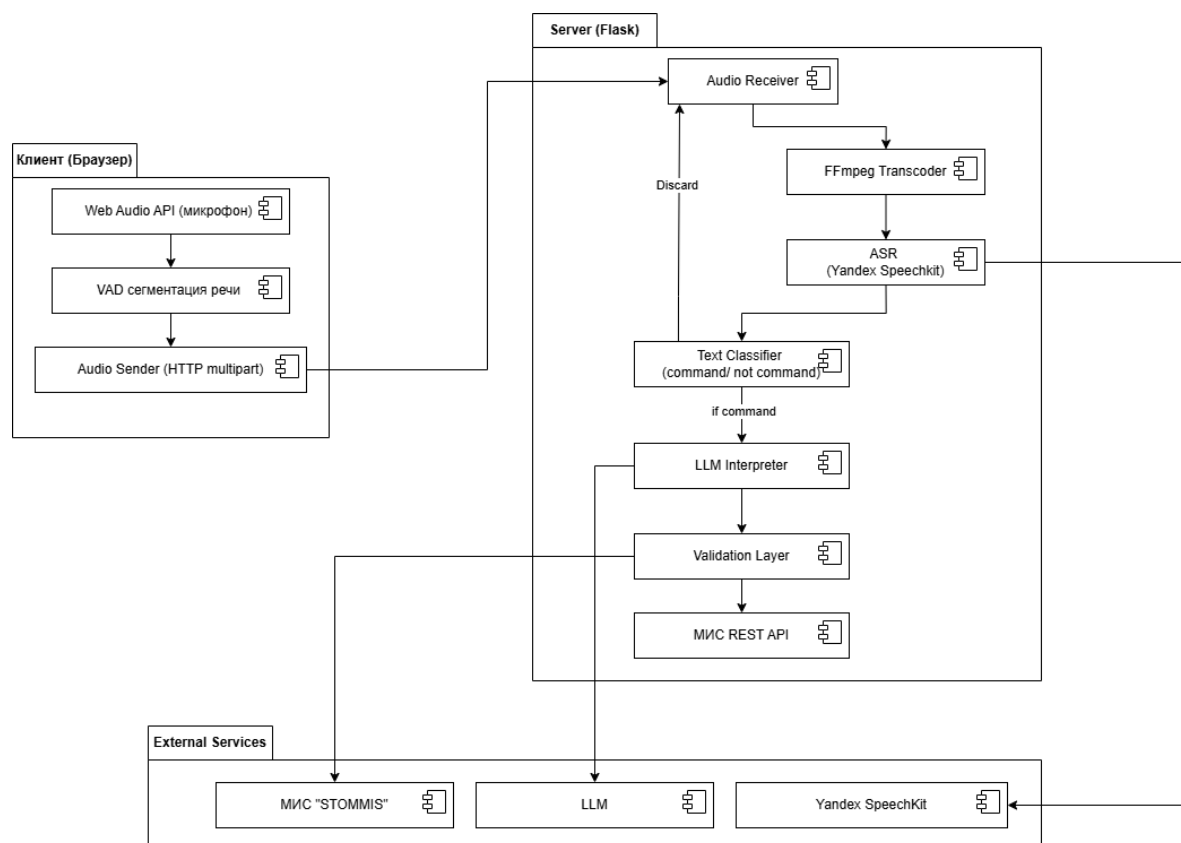


Рис. 2: Компонентная диаграмма подсистемы голосового управления

3.2. Сценарий взаимодействия

Основной сценарий работы системы показан на диаграмме последовательностей (рис. 3). Клиент непрерывно выделяет сегменты речи с помощью VAD и отправляет их на сервер в виде аудиофайла. На сервере выполняется распознавание речи (ASR), после чего применяется бинарный классификатор *command/not-command*, определяющий, требуется ли дальнейшая обработка. Если сегмент классифицирован как *not-command*, он отбрасывается и обработка завершается без обращения к LLM. Если сегмент классифицирован как *command*, транскрипция передаётся в модуль интерпретации (правила/LLM), который формирует структурированное действие. Далее выполняется валидация результата и внесение изменений в карту пациента посредством REST API МИС «СТОММИС». Механизм активации по ключевому слову (wake word) рассматривался как альтернативный подход, однако в итоговой архитектуре основным является метод классификации команд.

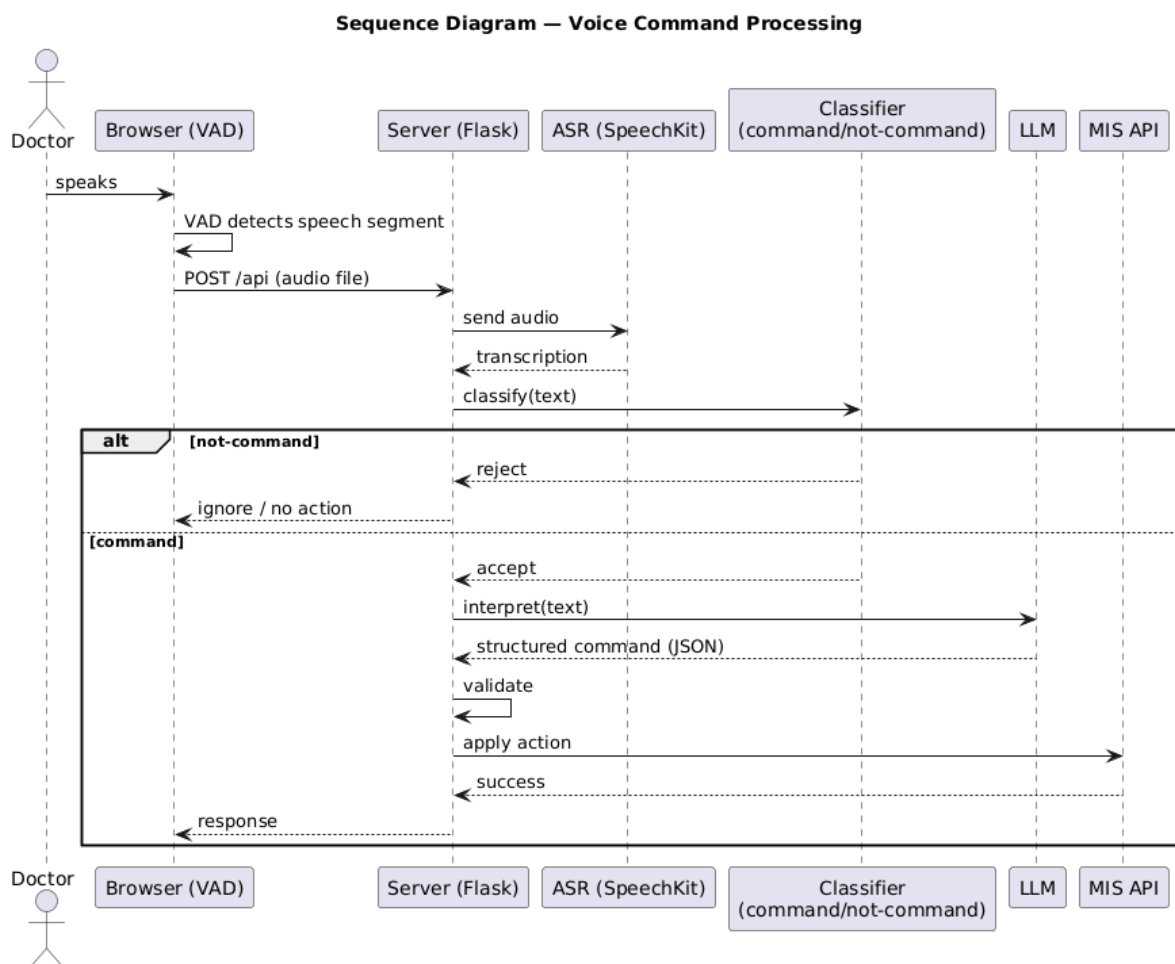


Рис. 3: Диаграмма последовательностей основного сценария обработки

Архитектурное разделение позволяет вынести вычислительно затратные этапы (ASR и интерпретацию) на сервер, сохранив на клиенте только ресурсоэффективный анализ аудиопотока и подготовку сегментов речи. Это снижает сетевую нагрузку и стоимость обработки по сравнению с подходом непрерывного распознавания всего аудиопотока.

3.3. Модуль интеллектуальной обработки команд

Модуль интеллектуальной обработки команд является ключевым элементом предлагаемой архитектуры и выполняет преобразование текстовой транскрипции в структурированные действия для МИС «СТОМ-МИС».

Модуль включает два основных компонента:

- **Бинарный классификатор *command/not-command***, который выполняет предварительную фильтрацию транскрипций и определяет, требуется ли дальнейшая обработка;
- **Модуль интерпретации (LLM)**, реализованный с использованием языковой модели, который преобразует команду в структурированное представление (intent и параметры).

Классификатор выполняет роль ресурсоэффективного фильтра, позволяя существенно снизить число обращений к языковой модели. Таким образом, достигается баланс между качеством интерпретации и вычислительными затратами.

4. Особенности реализации

В данном разделе описывается реализованный прототип подсистемы голосового управления для МИС «СТОММИС». Решение построено как конвейер: на стороне клиента в браузере выделяются речевые сегменты, а на сервере выполняются транскодирование (при необходимости), распознавание речи (ASR), затем фильтрация по транскрипции с использованием бинарного классификатора *command/not-command*, и только после этого — интерпретация команды в контролируемое действие внутри МИС.

4.1. Клиентский модуль: захват аудио и VAD

Клиентская часть реализована как веб-страница, получающая доступ к микрофону через Web Audio API [10]. Далее непрерывно выполняется детекция речевой активности (VAD) с использованием @ricky0123/vad-web [8], исполняющего нейросетевую модель в формате ONNX через ONNX Runtime Web [7]. VAD выдаёт события начала и окончания речи, а также оценку вероятности речи. Пример событий VAD-модели:

- `onSpeechStart` — фиксируется начало речевого сегмента;
- `onSpeechEnd` — сегмент считается завершённым после паузы (периода «тишины») заданной длительности;
- `onUpdate` — обновление вероятности речи (для визуализации и отладки).

Для завершения команды используется правило: «конец команды — 2 секунды тишины». Практически это реализуется параметром ожидания «тишины» после окончания речи (например, `redemptionFrames`, эквивалентный приблизительно двум секундам).

Пороговые параметры VAD настраиваются экспериментально под условия кабинета: `positiveSpeechThreshold` — порог включения речи (старт сегмента при превышении), `negativeSpeechThreshold` — порог

выключения (переход в тишину), а `redemptionFrames` задаёт сколько кадров тишины подряд требуется для фиксации окончания сегмента.

4.2. Клиентский модуль: выделение сегментов речи

Основная проблема непрерывного распознавания речи состоит в том, что на сервер может передаваться нерелевантная информация (общение с пациентом, фоновые реплики), что увеличивает нагрузку и стоимость обработки.

В предлагаемом решении клиент выполняет только выделение сегментов речи с помощью VAD, не принимая решения о том, является ли сегмент командой. Каждый завершённый речевой сегмент передаётся на сервер в виде аудиофайла.

Дальнейшее решение о необходимости обработки сегмента принимается на сервере после распознавания речи. Такое разделение ответственности позволяет упростить клиентскую часть и перенести более сложную логику фильтрации в серверный контур, где доступны текстовые данные и вычислительные ресурсы.

4.3. Wake word: варианты реализации и выбранный подход

В рамках работы были реализованы и проверены два подхода.

1) **Wake word в браузере (локальный KWS)**. Прототип выполнял обнаружение ключевого слова на клиенте по аудиопризнакам (на уровне коротких сегментов, выделенных VAD). Практические испытания показали две проблемы:

- высокая доля пропусков (*false negatives*) в реальных условиях кабинета;
- необходимость обучения/калибровки под каждого врача (разные голоса и микрофоны), что неудобно для внедрения.

Поэтому данный вариант не был принят как основной.

2) Wake word на сервере (проверка по ASR). В итоговой версии используется серверная проверка: короткий сегмент, выделенный VAD, распознаётся ASR, после чего выполняется текстовая проверка на наличие ключевого слова (например, «старт»). Это увеличивает число обращений к ASR в режиме ожидания, но снижает риск пропусков и не требует обучения под каждого врача. Фактически режим ожидания работает как «VAD + короткие ASR-проверки», а после успешной активации следующий сегмент речи рассматривается как команда.

Практические испытания показали, что регулярное использование wake word неудобно для врача в условиях реального приёма и снижает естественность работы. Поэтому в итоговой версии прототипа данный механизм рассматривается как резервный сценарий и базовая линия сравнения, тогда как основной подход использует автоматическую идентификацию команд по транскрипции после ASR.

4.4. Серверный модуль: приём аудио, ASR и интерпретация

Серверная часть реализована на Python (Flask) и предоставляет HTTP-endpoint для приёма командных аудиосегментов. Клиент отправляет запрос вида:

- метод POST;
- тело `FormData` с полем `file`;
- дополнительные поля метаданных (идентификатор врача, пациента, режим команд), если требуется контекст.

На сервере выполняются шаги.

1. проверка наличия поля `file` и чтение байтов аудио в память;
2. приведение формата аудио к поддерживаемому входу ASR: если MIME-тип отличается от ожидаемого, выполняется транскодирование в Ogg Opus через `ffmpeg` по пайпам (без записи на диск) [2];

3. распознавание речи посредством Yandex SpeechKit [12];
4. применение бинарного классификатора *command/not-command* к полученной транскрипции;
5. при классификации как *command* — интерпретация результата: преобразование текста в структурированную команду (например, *intent* + набор параметров), пригодную для заполнения полей карты пациента.

Интерпретация реализуется гибридно: для короткого списка команд задаётся допустимый набор интенгов, а для сопоставления свободных формулировок используется LLM-модуль. Результат возвращается клиенту в JSON и/или применяется на стороне МИС «СТОММИС».

4.5. Серверная идентификация команд по транскрипции

После распознавания речи (ASR) необходимо определить, является ли полученная транскрипция командой для системы или относится к речевому мусору. Для этого используется бинарный классификатор *command/not-command*.

В реализации классификатор построен на текстовых признаках транскрипции. Используются символьные n-граммы (char n-grams), извлекаемые с помощью TF-IDF, что обеспечивает устойчивость к ошибкам распознавания речи. Дополнительно учитываются признаки предметной области, такие как наличие номеров зубов, специфических медицинских терминов и характерных глаголов действий.

Модель классификации реализована как логистическая регрессия с балансировкой классов, что позволяет учитывать несбалансированность данных (команд существенно меньше, чем обычной речи).

Выбор бинарной постановки задачи обусловлен следующими причинами:

- задача фильтрации «команда/мусор» существенно проще полной интерпретации;

- модель может быть лёгкой и быстро выполняться;
- классификатор позволяет сократить число вызовов LLM и тем самым уменьшить вычислительную нагрузку и стоимость обработки.

При выборе порога решения приоритетом является высокая полнота (recall) класса *command*, чтобы минимизировать пропуски команд врача.

Обобщённая логика серверной обработки может быть представлена следующим образом:

```
audio -> ASR -> text
if not is_command(text):
    stop
else:
    result = LLM(text)
    validate(result)
    apply_to_STOMMIS(result)
```

4.6. Ограничение вывода языковой модели и валидация результата

Для интерпретации распознанного текста команды используется большая языковая модель `qwen3-235b-a22b-fp8`, доступная через совместимый с OpenAI API интерфейс (`base_url = https://rest-assistant.api.cloud.yandex.net/v1`, `project = folder_id`). Задача модели заключается не в генерации произвольного текста, а в преобразовании транскрипции команды в структурированное действие, пригодное для выполнения в МИС «СТОММИС».

Чтобы исключить некорректные или опасные действия, вывод модели жёстко ограничивается:

- допускается только формат **JSON** фиксированной структуры;

- тип действия выбирается только из заранее определённого множества интенгов (белый список);
- параметры команды (слоты) проверяются на корректность типов и принадлежность к допустимым значениям;
- при неоднозначности модель обязана вернуть статус `needs_clarification` вместо выполнения действия.

Структура результата (контракт). Результат интерпретации команды представляется объектом следующего вида:

```
{
  "intent": "set_diagnosis | set_tooth_state | add_procedure | ...",
  "slots": { ... },
  "confidence": 0.0..1.0,
  "needs_clarification": true|false,
  "clarification_question": "..."
}
```

Схема допустимого вывода. Для обеспечения воспроизводимости и проверки корректности используется схема (упрощённый вариант):

```
intent: enum[set\_diagnosis, set\_tooth\_state, add\_procedure,
  cancel, ...]
slots: object зависит( от intent)
confidence: number [0..1]
needs_clarification: boolean
clarification_question: string (optional, only if
  needs_clarification=true)
```

Пример интерпретации команды. Пусть врач произносит: «*Зуб 26: кариес. Добавь пломбу.*». После распознавания речи система получает текстовую транскрипцию и передаёт её в LLM вместе с контекстом

(текущий пациент, выбранный врач, активная карта, доступные справочники).

Ожидаемый результат интерпретации:

```
{
  "intent": "set_tooth_state",
  "slots": {
    "tooth": "26",
    "state": "caries",
    "procedure": "filling"
  },
  "confidence": 0.83,
  "needs_clarification": false
}
```

Выбор и конфигурация языковой модели. В прототипе для семантической интерпретации команд используется большая языковая модель `qwen3-235b-a22b-fp8`, доступная через API облачной платформы (вызов через совместимый клиент OpenAI с заданным `base_url`). Выбор данной модели обусловлен качеством понимания русскоязычных команд и устойчивостью к вариативным формулировкам при сохранении приемлемой задержки. Параллельно проводится исследование альтернативной модели `gpt-oss-20b`; на момент написания работы результаты сравнения являются предварительными, поэтому в основной конфигурации прототипа фиксируется модель `qwen3-235b-a22b-fp8`.

Ограничение допустимого вывода модели. Чтобы исключить неконтролируемые действия, в прототипе применяется управляемая генерация, при которой поведение LLM контролируется через три канала: **(1) инструкцию** (системное сообщение), **(2) ввод** (контекст, добавляемый к пользовательской команде), и **(3) инструменты** (tool-вызовы для получения допустимых значений из предметной области). Сервер принимает результат *только* при успешной валидации JSON по схеме и принадлежности значений разрешённым множествам.

Системная инструкция и формат вывода. Системная инструкция фиксирует назначение модели, запрещает свободный текст и задаёт формат результата согласно схеме из предыдущего пункта. Упрощённый фрагмент инструкции:

Ты - часть процедуры обработки данных первичного осмотра пациента.
Тебе будут приходить запросы вида:

1. Команда пользователя (распознанная речь врача)
2. Пронумерованный список вариантов обработки этой команды

Необходимо выбрать наилучшие варианты и вернуть результат в формате JSON:

```
{
  "schema": "http://json-schema.org/draft-07/schema",
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "chosen_option": { "type": "integer" },
      "V": { "type": "string" },
      "ABC": { "type": "string" },
      "Z": { "type": "string" },
      "to_delete": { "type": "integer" }
    },
    "required": ["id", "chosen_option", "V", "ABC", "Z", "to_delete"],
    "additionalProperties": False,
  }
}
```

Инструменты (tool) для зубной карты. Для сценариев заполнения зубной формулы LLM получает доступ к инструменту, возвращающему допустимые действия и параметры для конкретного клинического контекста (например, для выбранного зуба). Инструмент реализует поиск по доменному индексу допустимых команд и возвращает

структурированные данные, которые используются моделью для выбора `intent` и заполнения `slots`.

4.7. Интеграция с МИС «СТОММИС» и особенности предметной области

В отличие от решений, ориентированных на «протоколирование» консультации, в рассматриваемом сценарии цель — заполнение структурированной карты. Это требует:

- приведения распознанного текста к заранее определённым категориям (диагноз, манипуляция, зуб, рекомендация);
- сопоставления со справочниками (например, список процедур и заболеваний), используемыми в МИС «СТОММИС»;
- минимизации количества действий врача в интерфейсе.

Командный режим позволяет врачу произносить короткие инструкции, которые конвертируются в конкретные изменения в карте. Например, команда «старт, кариес на 26 зубе» может быть интерпретирована как выбор диагноза «кариес» и установка соответствующего зуба в формуле.

4.8. Нефункциональные требования: ресурсы

Решение спроектировано с учётом ограничений клиники:

- **Ресурсоэффективность:** постоянная работа VAD в браузере должна быть существенно дешевле постоянного ASR;
- **Минимизация трафика:** на сервер отправляются только короткие сегменты команд;

Перечисленные свойства проверяются экспериментально в разделе 6.

5. Интерпретация команд и заполнение медицинской карты

Данный этап соответствует модулю интеллектуальной обработки команд, описанному в разделе 3, и детализирует процесс преобразования транскрипции в структурированные действия системы.

После завершения этапа распознавания речи система получает текстовую транскрипцию голосовой команды врача. Однако прямое использование данного текста для заполнения медицинской документации является неэффективным и может приводить к ошибкам, поскольку речь врача часто содержит уточнения, комментарии и фразы, не предназначенные для прямого сохранения в медицинской карте.

В связи с этим в разрабатываемой системе реализован отдельный этап интерпретации команд, целью которого является преобразование неструктурированной текстовой информации в формализованное представление, пригодное для автоматизированного внесения данных в электронную медицинскую карту пациента.

Перед этапом интерпретации все полученные транскрипции проходят дополнительную фильтрацию.

На серверной стороне применяется бинарный классификатор *command/not-command* (см. раздел 4.5), который определяет, является ли данная фраза управляющей командой для системы.

Дальнейшая обработка выполняется только для фраз, классифицированных как команды, в то время как остальные речевые фрагменты отбрасываются как нерелевантные. Обобщённый алгоритм обработки команды можно представить следующим образом:

```
audio -> ASR -> text
if not command:
    stop
else:
    LLM -> structured command
    validate
```


apply to MIS

5.1. Разделение речевых фрагментов и управляющих команд

Речь врача в процессе приёма пациента может содержать как управляющие команды, адресованные системе, так и пояснительные комментарии, не имеющие прямого отношения к заполнению медицинской карты. Для корректной обработки голосового ввода необходимо различать данные типы речевых фрагментов.

В предлагаемом подходе речевая команда рассматривается как управляющее высказывание, содержащее указание на конкретное действие системы, например выбор диагноза, добавление симптома или заполнение определённого поля медицинской карты. Все остальные речевые фрагменты рассматриваются как нерелевантные для автоматической обработки и отбрасываются на данном этапе.

В рамках реализованного прототипа данное разделение выполняется автоматически с использованием бинарного классификатора по текстовой транскрипции, что позволяет отказаться от необходимости явной активации команд (например, с помощью слова-триггера) и повысить удобство использования системы.

5.2. Формирование структурного представления команды

Для обеспечения однозначной интерпретации команд врача используется структурное представление команды, включающее тип действия и набор параметров. Каждая команда преобразуется в формализованную структуру следующего вида:

(intent, параметры)

Тип команды определяет общее направление действия, например добавление записи, выбор значения или подтверждение состояния па-

пациента. Объект команды указывает на элемент медицинской карты, к которому относится действие, а параметры содержат дополнительные данные, необходимые для выполнения команды.

5.3. Использование языковой модели для интерпретации команд

Важно отметить, что языковая модель применяется только к тем речевым сегментам, которые уже прошли фильтрацию классификатором *command/not-command*. Таким образом, LLM используется исключительно для интерпретации команд, а не для обработки произвольной речи.

Для преобразования текстовой транскрипции в структурированное представление используется языковая модель, способная выполнять семантический анализ естественного языка. Языковая модель получает на вход текст команды и контекст текущего состояния медицинской карты пациента.

На основе анализа входных данных модель определяет наиболее вероятный тип команды и соответствующие параметры. Использование языковой модели позволяет учитывать вариативность формулировок команд, характерную для естественной речи, и снижает зависимость системы от строго фиксированного набора шаблонов.

5.4. Ограничение пространства решений и фильтрация команд

Для повышения надёжности работы системы и предотвращения некорректных действий языковая модель не имеет доступа к произвольному пространству решений. Вместо этого интерпретация команд выполняется в рамках заранее определённого множества допустимых действий и объектов.

В частности, выбор диагнозов, симптомов и процедур осуществляется исключительно из предопределённых медицинских справочников

и классификаторов. Такой подход позволяет снизить вероятность логических ошибок и обеспечивает соответствие результатов обработки требованиям медицинских информационных систем.

Дополнительно такой подход позволяет компенсировать возможные ошибки классификации и распознавания речи, поскольку некорректные или неполные команды не проходят этап валидации и не приводят к изменению данных.

5.5. Особенности заполнения стоматологической карты

Стоматологическая медицинская карта обладает высокой степенью структурированности и включает множество взаимосвязанных параметров, таких как состояние отдельных зубов, наличие патологий и выполненные процедуры. Ручной выбор соответствующих значений из иерархических списков является трудоёмкой задачей и отвлекает врача от лечебного процесса.

В разрабатываемой системе голосовые команды используются для навигации по структуре стоматологической карты и выбора необходимых элементов без прямого взаимодействия с пользовательским интерфейсом. Это позволяет врачу сосредоточиться на пациенте и существенно ускоряет процесс заполнения документации.

5.6. Интеграция с медицинской информационной системой

После формирования структурированного представления команды выполняется её интеграция с медицинской информационной системой. На данном этапе проверяется корректность параметров команды и соответствие текущему состоянию медицинской карты пациента.

При успешной проверке система формирует запрос на изменение соответствующих полей и применяет его к медицинской карте **посредством REST API МИС «СТОММИС»**. В случае возникновения

неоднозначностей или ошибок команда отклоняется, а врач получает уведомление о необходимости уточнения запроса.

5.7. Сравнение с традиционным подходом

В традиционных системах голосового ввода распознанная речь сохраняется в медицинской карте в виде текстового комментария без дальнейшей обработки. Такой подход не позволяет автоматизировать работу с данными и требует последующего ручного анализа.

В отличие от этого, предлагаемый в работе метод ориентирован на интерпретацию речи как источника управляющих команд. Результатом обработки является не текст, а структурированные изменения в медицинской карте, что обеспечивает более высокий уровень автоматизации и снижает нагрузку на медицинский персонал.

Кроме того, использование предварительной фильтрации команд (*command/not-command*) позволяет сократить число обращений к дорогостоящей семантической обработке и уменьшить вероятность ложных интерпретаций по сравнению с подходом, в котором обрабатывается весь распознанный текст.

5.8. Выводы по главе

В данной главе был рассмотрен алгоритмический подход к интерпретации голосовых команд врача и автоматизированному заполнению медицинской карты пациента. Использование языковой модели в сочетании с ограничением пространства допустимых решений позволяет обеспечить надёжную и контролируемую обработку команд в медицинской информационной системе.

6. Эксперимент

Цель экспериментальной части — оценить качество выделения речевых сегментов и команд, а также ресурсные и временные характеристики прототипа по сравнению с базовым подходом непрерывного распознавания аудиопотока. Дополнительно выполнена апробация в реальных условиях стоматологической поликлиники СПб ГБУЗ «СП №8»; результаты позволяют сделать предварительные выводы о применимости подхода.

6.1. Исследовательские вопросы

RQ1: Насколько уменьшается объём аудио, отправляемого на сервер, и связанная с этим стоимость распознавания по сравнению с непрерывным ASR?

RQ2: Каково качество выделения и распознавания команд (precision/recall) при использовании VAD и последующей серверной фильтрации транскрипции с помощью классификатора *command/not-command*?

RQ3: Как влияет настройка параметров VAD (пороги, длительность «тишины» до завершения) на ложные срабатывания и задержку?

RQ4: Какова суммарная задержка «команда → действие в МИС» и какие компоненты конвейера вносят основной вклад?

6.2. Метрики

Для оценки использовались следующие метрики:

- **Latency** (мс): время от окончания произнесения команды до получения результата на клиенте; отдельно фиксируются компоненты: сегментация, передача, ASR, интерпретация, применение действия.

- **Нагрузка клиента:** влияние на отзывчивость UI и оценка потребления CPU при включенном VAD.
- **Нагрузка сервера:** число обращений к ASR и средняя длительность обрабатываемых аудиосегментов.
- **Качество классификации:** precision, recall и F_1 для бинарной классификации *command/not-command*;
- **Снижение числа вызовов LLM:** доля сегментов, отфильтрованных классификатором до этапа интерпретации;
- **Качество сегментации:** доля корректно выделенных командных сегментов (по ручной разметке на небольшом наборе записей).

6.3. Сценарии и базовая линия сравнения

Рассматривались два режима:

1. **Базовый режим (baseline):** непрерывная запись и распознавание всего аудиопотока приёма (или всех сегментов речи), без VAD-фильтрации на клиенте.
2. **Предложенный режим:** непрерывный VAD в браузере (сегментация речи) и серверная фильтрация транскрипции с использованием классификатора *command/not-command*; на сервер отправляются только речевые сегменты, а дальнейшая обработка выполняется только для сегментов, классифицированных как команды.

6.4. Результаты и обсуждение

Ключевые результаты сведены в табл. 2. В рамках пилотной апробации наблюдаемая задержка от конца произнесения команды до получения структурированного результата на сервере составляет порядка 2–3 секунд; значения зависят от качества сети и акустической обстановки

и требуют дальнейшей стабилизации. Численные значения зависят от условий (микрофон, шум, сеть).

В рамках эксперимента дополнительно фиксировалась доля сегментов, отфильтрованных классификатором до этапа интерпретации, что позволяет оценить снижение нагрузки на LLM и связанные с этим вычислительные затраты.

Как видно из табл. 3, классификатор обеспечивает высокую полноту (recall) для класса команд, что критично для практического использования, поскольку пропуски команд влияют на удобство врача сильнее, чем ложные срабатывания. При этом значительная доля речевых сегментов отфильтровывается до этапа интерпретации, что снижает нагрузку на LLM.

Для повышения реалистичности в дальнейшем планируется использование реальных записей врачебной речи и ручной разметки команд.

Матрица ошибок показывает, что основная часть ошибок приходится на ложные срабатывания (FP), что является допустимым компромиссом, поскольку система в этом случае лишь выполняет дополнительную проверку, но не теряет реальные команды врача.

6.4.1. RQ1: ресурсоёмкость и стоимость

Ожидается, что предложенное решение значительно уменьшит T_{sent} и сетевой трафик за счёт исключения пауз и нерелевантной речи. Дополнительно фиксируется загрузка CPU в браузере: в режиме мониторинга должна наблюдаться существенно меньшая нагрузка, чем при постоянной записи/кодировании и передаче аудио.

6.4.2. RQ2: качество команд

Для качества выделения команд важно учитывать два типа ошибок: ложные срабатывания (речевой сегмент классифицирован как команда, но таковой не является) и пропуски (реальная команда классифицирована как *not-command*).

Оценка проводится на размеченных сценариях и отражается через

precision, recall и F_1 для задачи бинарной классификации. Приоритетом является высокая полнота (recall) класса *command*, чтобы минимизировать пропуски команд врача.

6.4.3. RQ3: влияние параметров

Проводится серия запусков при различных значениях порогов VAD и длительности паузы завершения. Анализируется компромисс между задержкой (слишком длинная пауза увеличивает L) и устойчивостью (слишком короткая пауза может преждевременно обрывать команду).

6.4.4. RQ4: задержка и нагрузка

Проводится серия замеров задержки и нагрузки на сервере при различных длительностях команд и числе обращений к ASR. Анализируется влияние параметров VAD и классификатора на общее число обрабатываемых сегментов и нагрузку на систему.

В ходе апробации в СПб ГБУЗ «СП №8» было отмечено, что основной источник нестабильности связан с условиями акустики и вариативностью речи врачей. Это влияет как на качество распознавания речи, так и на точность классификации команд.

Использование серверной фильтрации транскрипции с помощью классификатора *command/not-command* позволило снизить количество нерелевантных обращений к модулю интерпретации и повысить устойчивость системы по сравнению с подходом, основанным на явной активации по ключевому слову.

6.5. Обсуждение результатов

При интерпретации результатов важно учитывать особенности стоматологического кабинета: наличие фоновой речи, инструмента и оборудования, а также различия в манере диктовки у разных врачей. Планируется отдельно анализировать типовые причины ошибок (например, смешение речи врача и пациента, тихая речь, высокая шумовая нагрузка) и влияние этих факторов на качество системы.

6.6. Угрозы нарушения корректности

Основные угрозы валидности эксперимента:

- **Разметка:** неточность временных границ команд и субъективность в определении «команды».
- **Сеть и инфраструктура:** задержки в сети могут влиять на измеряемую латентность.

Таблица 2: Сравнение базового и предложенного режимов (оценка на типовом сценарии)

Показатель	Baseline (непрерывное распознавание)	Предложенный режим (VAD + классификация команд)
Передаваемая длительность аудио за 10 минут приёма	≈600 с (весь поток)	≈18–30 с (6 команд по 3–5 с)
Число обращений к ASR	высокое (поток/чанки)	низкое (по подтверждённым сегментам)
Средняя длина одного распознаваемого фрагмента	5–15 с (чанки потока)	3–5 с (командный сегмент)
Задержка «конец команды → действие в МИС»	распознавание идёт постоянно; ресурсы/стоимость — высокие	распознавание запускается только для речевых сегментов, классифицированных как команды; ресурсы/стоимость — низкие
Риск обработки нерелевантной речи	высокий (комментарии/шум идут в ASR)	сниженный (фильтрация VAD и классификатор)
Требования к взаимодействию врача	требуется управление записью/режимом	голосовое управление во время лечения (без рук)
Оценка стоимости ASR (отн.)	1.0x (весь поток)	0.03–0.05x (только команды)

Таблица 3: Качество классификации команд (оценка на тестовом наборе)

Метрика	Значение
Precision (command)	0.82
Recall (command)	0.90
F_1 -score	0.86
Доля отфильтрованных сегментов	≈ 70–80%

Таблица 4: Матрица ошибок классификатора (пример)

	Предсказано: команда	Предсказано: не команда
Факт: команда	TP = 45	FN = 5
Факт: не команда	FP = 10	TN = 40

7. Заключение

В рамках практики была разработана и исследована подсистема голосового управления для МИС «СТОММИС» в части заполнения зубной формулы и результатов первичного осмотра пациентов, ориентированная на использование голосовых команд в процессе лечения без необходимости постоянного взаимодействия с интерфейсом. Ключевая идея решения — многоэтапная обработка аудио: лёгкая сегментация речи в браузере (VAD), что снижает вычислительную и стоимостную нагрузку по сравнению с непрерывным распознаванием всего потока.

В ходе работы также рассматривались альтернативные способы активации голосового режима, включая ручную активацию (*push-to-talk*) и активацию по ключевому слову (*wake word*). Практические ограничения данных подходов послужили основанием для перехода к автоматическому выделению команд на основе классификации транскрипции.

Основные результаты работы, соответствующие поставленным задачам:

- выполнен обзор существующих решений в области голосового управления, выделения речевых сегментов, идентификации команд и интерпретации распознанного текста; по результатам обзора обоснован выбор архитектуры с VAD в браузере, ASR на сервере и фильтрацией команд по транскрипции;
- спроектирована архитектура подсистемы голосового управления для МИС «СТОММИС»: определены основные компоненты, границы ответственности, основной сценарий взаимодействия и место модулей интеллектуальной обработки команд в общем конвейере;
- реализован клиентский модуль захвата аудио и детекции речи в браузере с использованием Web Audio API и VAD на базе @ricky0123/vad-web / ONNX Runtime Web;
- реализована логика выделения речевого сегмента: правило окончания по паузе (например, 2 секунды тишины) и передача сегмента на сервер в формате `multipart/form-data` с полем `file`;

- реализован серверный конвейер обработки: приём `file`, транскодирование аудио (FFmpeg → Ogg Opus), распознавание речи через Yandex SpeechKit, классификация транскрипции с помощью бинарного классификатора *command/not-command*, интерпретация команды и применение результата к карте через REST API МИС «СТОММИС»;
- реализован безопасный слой интерпретации команд: использование LLM `qwen3-235b-a22b-fp8` с ограничением вывода (JSON-контракт, whitelist интенгов/слотов, валидация, режим `needs_clarification`); проведён предварительный анализ альтернатив, включая `gpt-oss-20b`;
- проведена экспериментальная оценка и пилотная апробация в СПб ГБУЗ «СП №8»; получены предварительные результаты по задержке, снижению объёма обрабатываемого аудио и качеству классификации команд, при этом требуется дальнейшая стабилизация в условиях шума и вариативной речи врачей.

Направления дальнейшей работы включают расширение набора команд, улучшение устойчивости к шумам, а также уточнение механизма ограничения допустимых действий при интерпретации команд (в частности, формализацию схемы ответа и справочников предметной области). Полученные результаты подтверждают возможность практического применения предложенного подхода для автоматизации работы врача в медицинских информационных системах с использованием методов машинного обучения и языковых моделей без усложнения пользовательского взаимодействия.

Список литературы

- [1] AI Mycroft, contributors. openWakeWord: open-source wake word detection. — 2025. — URL: <https://github.com/dscripka/openWakeWord> (дата обращения: 2026-01-04).
- [2] FFmpeg Developers. FFmpeg Documentation. — 2025. — URL: <https://ffmpeg.org/documentation.html> (дата обращения: 2026-01-04).
- [3] Но Ricky. @ricky0123/vad-web: User Guide and API. — 2025. — URL: <https://docs.vad.ricky0123.com/user-guide/api/> (дата обращения: 2026-01-04).
- [4] MDN Web Docs. MediaDevices.getUserMedia(). — 2025. — URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia> (дата обращения: 2026-01-04).
- [5] MDN Web Docs. MediaRecorder API. — 2025. — URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder> (дата обращения: 2026-01-04).
- [6] MDN Web Docs. Web Audio API. — 2025. — URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API (дата обращения: 2026-01-04).
- [7] Microsoft. ONNX Runtime Web. — 2025. — URL: <https://onnxruntime.ai/docs/get-started/with-javascript.html> (дата обращения: 2026-01-04).
- [8] Ricky0123. @ricky0123/vad-web: Voice Activity Detection for the Web. — 2024. — URL: <https://docs.vad.ricky0123.com/> (дата обращения: 2026-01-05).
- [9] Silero Team. Silero VAD: pre-trained Voice Activity Detection models. — 2025. — URL: <https://github.com/snakers4/silero-vad> (дата обращения: 2026-01-04).

- [10] W3C. Web Audio API Specification. — <https://www.w3.org/TR/webaudio/>. — 2024. — Дата обращения: May 12, 2026.
- [11] WebRTC Project. WebRTC Native Code: Voice Activity Detector (VAD). — 2024. — URL: https://webrtc.googlesource.com/src/+refs/heads/main/common_audio/vad/ (дата обращения: 2026-01-05).
- [12] Yandex Cloud. SpeechKit: распознавание и синтез речи. — 2025. — URL: <https://cloud.yandex.ru/services/speechkit> (дата обращения: 2026-01-04).